

Evaluación Bases de Datos: Diseño de Esquema

Trabajo Práctico Intermedio

Curso de Backend Developer

Objetivo General

Diseñar el modelo de datos de la aplicación que vas a construir durante el resto de la cursada. Esta evaluación es **de diseño, no de código**: todavía no vimos Express, así que lo que se entrega es el plano de datos sobre el que después vas a programar la API.

El dominio del negocio lo elegís vos. No tiene que ser un e-commerce — puede ser un sistema de turnos, una plataforma de publicaciones, un catálogo de alojamientos, una gestión de eventos, lo que quieras — siempre y cuando cumpla con las entidades obligatorias descriptas abajo.

Entidades Obligatorias

El esquema tiene que incluir, como mínimo, estas tres piezas. Los nombres son orientativos: adaptalos a tu dominio.

Entidad	Rol en el esquema	Ejemplos según el dominio
Usuarios	Quién usa el sistema. Debe tener un campo que permita diferenciar roles (ej. admin vs. cliente).	Usuarios, Clientes, Miembros
Entidad Principal	El corazón del negocio: lo que la aplicación gestiona.	Producto, Publicación, Turno, Alojamiento, Evento
Entidad Referenciada	Una colección aparte a la que la Entidad Principal hace referencia (no se embebe).	Categoría, Rubro, Tipo, Departamento

Importante: Las tres entidades de arriba son obligatorias, sea cual sea el dominio elegido. El resto del modelo (entidades adicionales, campos extra) queda a tu criterio, siempre que se sostenga la relación de referencia entre la Entidad Principal y la Entidad Referenciada.

Consigna

1. Elección del dominio

Definí en una línea de qué se trata tu aplicación y quién la usa. Esto va a ser también la base del proyecto que vas a implementar en los próximos módulos.

2. Diagrama del modelo

Representar las colecciones y sus relaciones (podés usar dbdiagram.io, MongoDB Compass Schema, draw.io, o incluso a mano/foto, siempre que se entienda). Debe mostrarse:

- Cada colección con sus campos principales.
- El tipo de relación entre Entidad Principal y Entidad Referenciada (1:N).
- Cualquier otra relación que hayas agregado (ej. Usuario ↔ Entidad Principal).

3. Justificación de embeber vs. referenciar

Este es el punto central del ejercicio. Por escrito, explicar:

- Por qué la Entidad Referenciada va en una colección aparte y no embebida dentro de la Entidad Principal.
- Si en algún caso decidiste embeber algo en lugar de referenciar, justificar por qué (ej. datos que no se reutilizan en ningún otro lado y siempre se leen junto con su padre).

4. Índices propuestos

Listar los índices que le pondrías a tu base y, para cada uno, una frase de para qué sirve (ej. índice único en el email de usuario para no permitir duplicados; índice en el campo de referencia de la Entidad Principal para acelerar los filtros por categoría).

5. Documentos de ejemplo

3 a 5 documentos JSON de ejemplo por colección, coherentes entre sí (que las referencias entre documentos existan de verdad). Esto es un adelanto de lo que después va a ir en el README del proyecto final.

Detalle de la Entrega

1. Un documento (PDF o Markdown) que contenga las 5 secciones de la consigna.
2. El diagrama puede ir embebido en el mismo documento o como archivo aparte (imagen o link).
3. Entrega a través del repositorio de GitHub que vas a usar para el proyecto final, o por el canal que indique el docente.

Aclaraciones

- No se evalúa código ni implementación — solo el diseño y su justificación.
- El esquema que entregues acá es el que vas a implementar en Mongoose a partir del Módulo 4, así que pensalo con la aplicación final en mente.
- Si durante el desarrollo de los próximos módulos necesitás ajustar algo del modelo original, es normal y esperable: la idea es que llegues con una base sólida, no con un diseño cerrado e inamovible.

Criterios de evaluación

1. Elección del dominio

Dominio definido en una línea, cualquier rubro (no tiene que ser e-commerce).

Las tres entidades obligatorias (Usuarios, Entidad Principal, Entidad Referenciada) están identificadas y aplicadas con sentido al dominio elegido.

Mínimo 3 tablas/colecciones en total.

2. Correctitud del modelo de datos

Cada tabla tiene sus atributos definidos.

Usuarios tiene un campo que diferencia roles.

Existe al menos una relación de referencia real entre una tabla y otra (Entidad Principal → Entidad Referenciada, vía algo equivalente a un `_id/ObjectId`).

Si hay relaciones adicionales (ej. Usuario ↔ Entidad Principal), también están bien planteadas.

3. Tipos de datos nativos

Cada atributo tiene declarado su tipo de dato nativo (los que soporta Mongo/Mongoose): String, Number, Boolean, Date, Array, Object (embebido), ObjectId (referencia).

El tipo elegido es coherente con el dato que representa (ej. boolean para flags, array para listas de valores, ObjectId para la referencia obligatoria).

4. Calidad del diagrama

Se entiende sin explicación adicional.

Muestra las tablas, sus campos y la(s) relación(es) con su cardinalidad (1:N como mínimo).

5. Justificación de la referencia

Explica por qué esa relación se modela como referencia (no hace falta discutir embeber, pero sí justificar por qué se referencia: reutilización, independencia de actualización, evitar duplicación, etc.).

6. Documentos de ejemplo

3 a 5 documentos JSON por colección.

Consistencia referencial real: los IDs referenciados existen efectivamente en la otra colección.

Los documentos respetan los tipos de datos y campos declarados en el diagrama.

7. Formato y entrega

Documento único (PDF/MD) con las secciones pedidas.

Subido al repo de GitHub del TP final, dentro de una carpeta DOCS.

El modelo entregado es el que efectivamente se va a implementar en Mongoose/Express más adelante (coherencia hacia adelante con el proyecto final).